Trabalho Prático — Cena 2D Interativa

Computação Gráfica

Atenção: O trabalho pode ser realizado **individualmente ou em dupla**. Entregue as atividades pelo Moodle. Todos os alunos poderão ser aleatoriamente convocados pelo professor para a **explicação e defesa do código entregue**, o que poderá afetar diretamente o conceito final da atividade.

1 Objetivo

Desenvolver um programa gráfico interativo em Python, podendo ser estruturado em Orientação a Objetos (classes), que renderize uma cena bidimensional. O programa deve ser construído expandindo a estrutura base do exemplo do "Sr. Coração" visto em aula, utilizando as bibliotecas `glfw, moderngl, numpy, pyglm` ou `pillow`.

2 Especificações da Cena e Interações

O programa deve cumprir, obrigatoriamente, os seguintes requisitos visuais e lógicos:

- **Composição da Cena:** A cena deve conter, diversos elementos, tais como uma casa, uma árvore, um sol, Sr. Coração, etc.
- **Múltiplas Primitivas:** É obrigatória a utilização de mais de um tipo de primitiva do OpenGL na construção dos elementos (por exemplo, misturar `moderngl.TRIANGLES` com `moderngl.TRIANGLE_FAN` ou `LINE_LOOP`).
- É necessário o reaproveitamento de geometrias (como no exemplo do Sr. Coração).
- **Alternância Dia/Noite:** O usuário deve poder alternar o estado global da cena, por ex. dia e noite, sol e chuva, por meio de uma tecla de atalho. Essa transição deve:
  1. Alterar a cor do fundo (utilizando `ctx.clear`).
  2. Atenuar a cor dos elementos renderizados, aplicando um fator de atenuação ao *shader* por meio de uma variável `uniform`.
- **Alteração Dinâmica de Cores:** O programa deve permitir alterar as cores de objetos específicos da cena em tempo real (por meio de botões ou teclas dedicados).
- **Movimentação (Setas do Teclado):** Deve haver interação mecânica com os elementos da cena por meio das setas do teclado.

3 Restrições Técnicas

- **OpenGL Core Profile:** O código deve utilizar estritamente o pipeline programável (OpenGL 4.0 Core). **É terminantemente proibido** o uso de recursos de OpenGL legado (*immediate mode*, `glBegin`, `glEnd`, matrizes embutidas, etc.).
- **Shaders:** A utilização de *shaders* personalizados é obrigatória. Toda a lógica de renderização e de cor deve vir dos arquivos `.vert` e `.frag` escritos em GLSL.
- **Bibliotecas Permitidas:** É permitido utilizar apenas o que já está instalado no *environment* inicial do curso (ex: `moderngl`, `glfw`, `numpy`).
- **Proibição de Wrappers:** É proibido o uso de bibliotecas de alto nível que encapsulem o GLSL ou o OpenGL (como Pygame, PyOpenGL legado, bibliotecas de cena prontas, etc.).
- **Exceções:** Caso o grupo deseje utilizar alguma biblioteca extra para um fim específico (ex: ler um tipo de arquivo diferente), deverá procurar o professor para validação e autorização prévias.

4 Entregáveis e Relatório

A entrega deverá ser submetida no Moodle (em formato `.zip`) contendo rigorosamente os seguintes itens:

1. **O Código-Fonte Completo:** Todos os arquivos necessários para rodar o projeto sem erros (`.py`, arquivos `.frag`, `.vert` e eventuais texturas/dados).
2. **Relatório do Projeto (PDF):** Um documento formal explicando o desenvolvimento, contendo:
    - **Explicação da Arquitetura:** Explicação textual de como o código foi estruturado, como as classes se comunicam e como os *shaders* foram pensados.
    - **Screenshots:** Capturas de tela que evidenciem a execução do programa (mostrando claramente o cenário no Modo Dia e no Modo Noite).
    - **Divisão de Tarefas (se em dupla):** É obrigatório descrever detalhadamente a contribuição efetiva de cada um dos dois participantes na produção do código e do relatório.
    - **Declaração de Uso de IA:** Caso a dupla/aluno tenha utilizado ferramentas de Inteligência Artificial (como GitHub Copilot, ChatGPT, Claude, etc.) como copiloto ou para auxiliar em alguma etapa de lógica ou de redação, **deve-se declarar explicitamente** qual ferramenta foi utilizada e para qual fim. Ressalta-se que, ao submeter o trabalho, o autor assume **total responsabilidade técnica e intelectual** pelo material (e eventuais bugs ou violações arquiteturais) gerado pela IA.